SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1739**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 1 **Duration : 1 Hour Total Marks:10**

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **(C. Amareswar Reddy/192412470)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. Amareswar Reddy

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

   i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:-

1. Problem Statement

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) using **Backtracking Search**.

2. Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

3. Backtracking Search Algorithm

Backtracking assigns a shift to one doctor at a time. If a constraint is violated, it rejects the assignment and tries another possibility.

Program code:-

```
def backtrack(doctors, shifts, assignment):
    if len(assignment) == len(doctors):
        return assignment
    doctor = doctors[len(assignment)]
    for shift in shifts:
        if shift not in assignment.values():
            assignment[doctor] = shift
            if is_valid(assignment):
                result = backtrack(doctors, shifts, assignment)
```

```
            if result:
                return result
        del assignment[doctor]
    return None
def is_valid(assignment):
    if "D1" in assignment and assignment["D1"] == "Night":
        return False
    if "D3" in assignment and assignment["D3"] == "Morning":
        return False
    if "D2" in assignment and "D3" in assignment:
        order = {"Morning": 1, "Afternoon": 2, "Night": 3}
        if order[assignment["D2"]] >= order[assignment["D3"]]:
            return False
    return True
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
result = backtrack(doctors, shifts, {})
if result:
    print("Valid Doctor Shift Assignment:")
    for doctor, shift in result.items():
        print(doctor, ":", shift)
else:
    print("No valid assignment found.")
```

4. Step-by-Step Execution

Step 1: Assign D1

- Try D1 = Morning.
- Constraint check: D1 cannot work Night. Morning is allowed.
- Assignment: D1 = Morning. ✓

Step 2: Assign D2

- Try D2 = Afternoon.
- Constraint check: D2 must work before D3.
- D2 = Afternoon requires D3 to work Night.
- D3 cannot work Morning, so Night is allowed.
- Assignment: D2 = Afternoon. ✓

Step 3: Assign D3

- Try D3 = Night.

- Constraint check: D3 cannot work Morning. ✓
- D2 works Afternoon and D3 works Night, so D2 works before D3. ✓
- Only one doctor is assigned to each shift. ✓

5. Output:-

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

6. Constraint Verification

Constraint	Verification	Status
D1 cannot work Night	D1 works Morning	✓ Satisfied
D2 must work before D3	Afternoon < Night	✓ Satisfied
Only one doctor per shift	Each shift has one doctor	✓ Satisfied
D3 cannot work Morning	D3 works Night	✓ Satisfied
Every doctor gets exactly one shift	All doctors are assigned	✓ Satisfied

7. Conclusion

Backtracking Search successfully finds a valid schedule by assigning shifts one at a time and checking the constraints. The final schedule is **D1 – Morning, D2 – Afternoon, and D3 – Night**. All constraints are satisfied.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly

Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided